

Experimental Notes for Rebuttal

Scaling up Measurement-Noise Scaling Laws

June 2026

Contents

1	Finding the Data	2
2	Geneformer Validation Loss Scaling	3
3	Geneformer Training Dynamics	5
4	Geneformer Model-Capacity Scaling	6
5	STATE Hyperparameter Sweep	7
6	MI Estimator Robustness: KSG vs. LMI	8
7	Linear Probe Scaling Curves	9
8	Learning Trajectory: MI at Each Training Checkpoint	10
9	Fine-Tuned Large Pretrained STATE	11

1 Finding the Data

All code, data pointers, and results described in this document live in the following repository:

<https://github.com/igor-sadalski/Scaling-up-measurement-noise-scaling-laws.git>

The experiments described here were run on the **adding-state** branch. Raw data and trained model checkpoints are not stored in the repository itself; they live on S3 at `s3://measurement-noise-scaling-laws/data/` and can be accessed locally via the `S3Retriever` utility in `scaling_laws/s3_retriever.py`. The local data root defaults to `~/noise_scaling/data/` and can be overridden with the `NOISE_SCALING_DATA_DIR` environment variable. All result CSV and PNG files referenced throughout this document are committed to the repository under `analysis/final_results/`.

2 Geneformer Validation Loss Scaling

Geneformer was implemented as a BERT-style masked-language model (`BertForMaskedLM`) using the HuggingFace Trainer. The control architecture used throughout has 256 hidden dimensions, 4 attention heads, 3 transformer layers, a 512-dimensional feedforward (intermediate) layer, maximum input length of 512 tokens, and attention and hidden dropout of 0.02. Training used a masked-language-modelling objective with 15% token masking, AdamW optimiser, learning rate 10^{-3} , weight decay 0.001, 5 000 warmup steps with a linear decay schedule, and a per-device batch size of 64. Early stopping monitored validation loss with patience 5; checkpoints were saved every 1 000 optimiser steps. Experiments were run across a 10×10 grid of cell counts and quality levels for all four datasets (PBMC, larry, MERFISH, shendure), yielding 400 experiments per dataset. After training, the saved best-checkpoint model was loaded and `GeneformerPretrainer.evaluate()` was run on the held-out `test` split (separate from the validation split used during training), producing a masked-language-modelling cross-entropy loss saved to `test_loss.txt`. This is a fresh inference pass on unseen data, not the validation loss recorded in the training log. These 400 scalar values were plotted on a log-log scale and a power law $\mathcal{L}(N) \propto N^{-\alpha}$ was fitted independently for each quality level using ordinary least squares on the log-transformed values.

Data: `analysis/final_results/geneformer_loss_scaling.csv`

Geneformer Test Loss vs Number of Cells
Y-AXES INVERTED (better $\uparrow$)

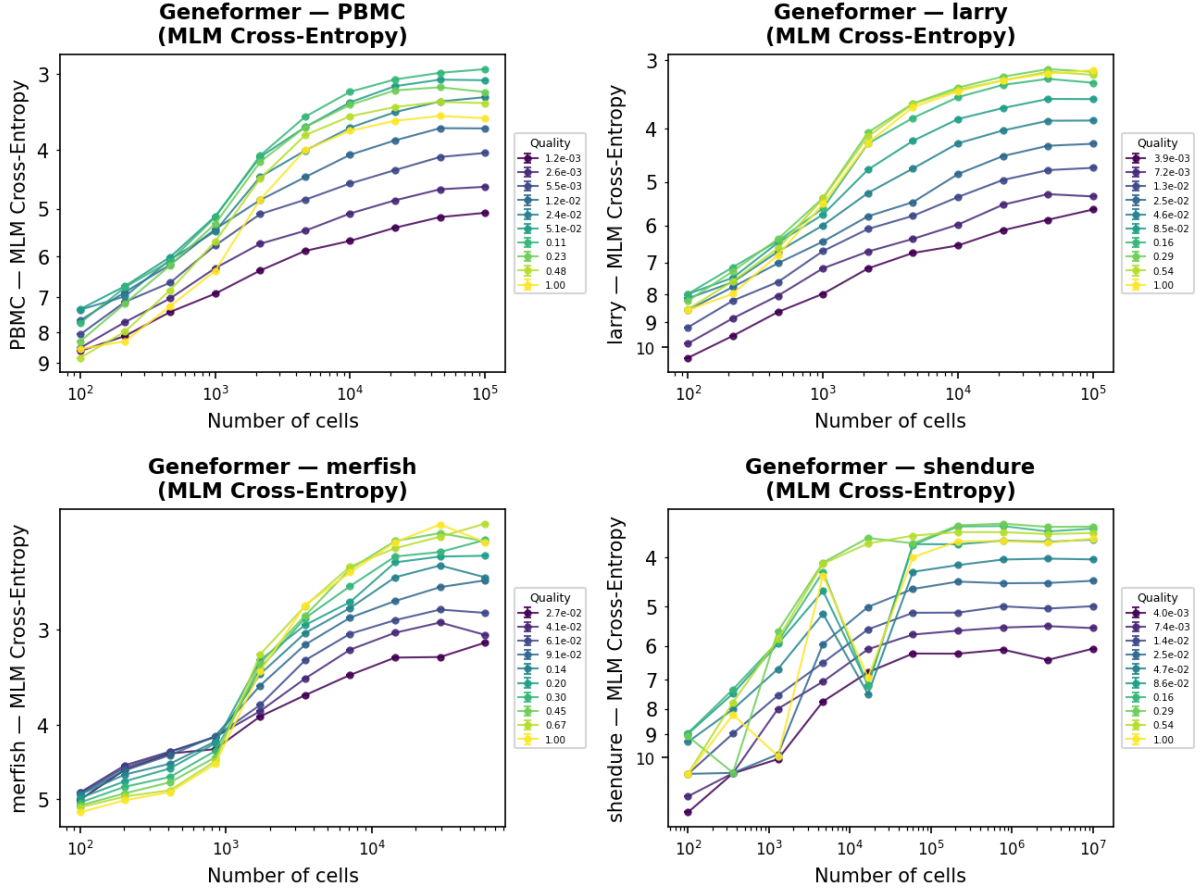


Figure 1: Geneformer final validation loss vs. training-set size on a log-log scale, with power-law fits per quality level.

3 Geneformer Training Dynamics

Training and validation loss trajectories were collected for every Geneformer experiment using the same architecture and training protocol described in Section 1 (256 hidden, 4 heads, 3 layers, 512 FFN, $\text{lr} = 10^{-3}$, batch size 64, early stopping patience 5). Loss histories were read from the `log_history` field of `trainer_state.json`; each entry records the global step, training loss, and evaluation loss. To keep the figure readable, each trajectory was subsampled to 50 data points selected by evenly-spaced row index (not by step value), and the best checkpoint — identified from the `best_model_checkpoint` field written by the HuggingFace Trainer into `trainer_state.json` — was marked with a cross. Three size ranks — the smallest, a middle, and the largest available cell count — were selected per dataset and plotted together in a single multi-panel figure. Early-stopping patience for Geneformer was 3 evaluation rounds (i.e. 3000 steps without improvement).

Data: `analysis/final_results/geneformer_loss_curves_sml.csv`

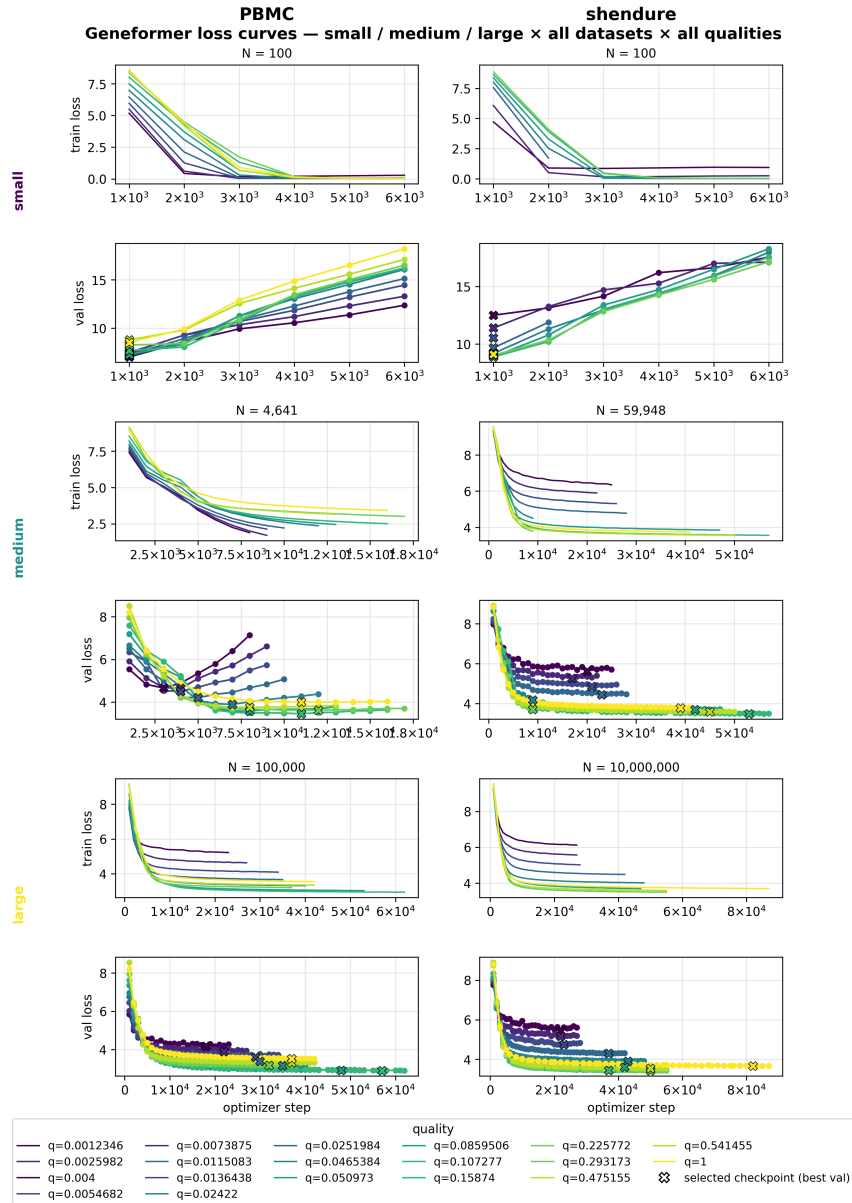


Figure 2: Geneformer training (solid) and validation (dashed) loss curves for three size ranks across all datasets. Crosses mark the early-stopping checkpoint.

4 Geneformer Model-Capacity Scaling

Seven Geneformer configurations were trained on PBMC at the largest available cell count (100 000 cells) across all ten quality levels. The configurations span a range of roughly 0.012M to 56.62M body parameters by jointly scaling embedding dimension, number of layers, number of attention heads, and feedforward width:

Embed dim	Layers	Heads	FFN	Max input	Body params (M)
32	1	1	64	512	~0.012
64	2	1	128	512	~0.098
128	2	2	256	512	~0.39
256	3	4	512	512	~2.36
384	4	6	768	512	~7.08
512	6	8	1024	512	~18.87
768	8	12	1536	512	~56.62

Table 1: Geneformer architecture sweep configurations. The 256-dim row (control) matches the configuration used in all other experiments.

All configurations shared the same training hyperparameters: AdamW, $\text{lr} = 10^{-3}$, weight decay 0.001, 5 000 warmup steps, linear schedule, batch size 64, 40 000 total optimiser steps, early stopping patience 5 (eval every 1 000 steps, metric: `eval_loss`). After training, cell embeddings were extracted with `EmbExtractor` and LMI was computed on the `protein_counts` signal. MI was plotted against total trainable parameter count on a log-log scale.

Data: `analysis/final_results/geneformer_model_size_sweep.csv`

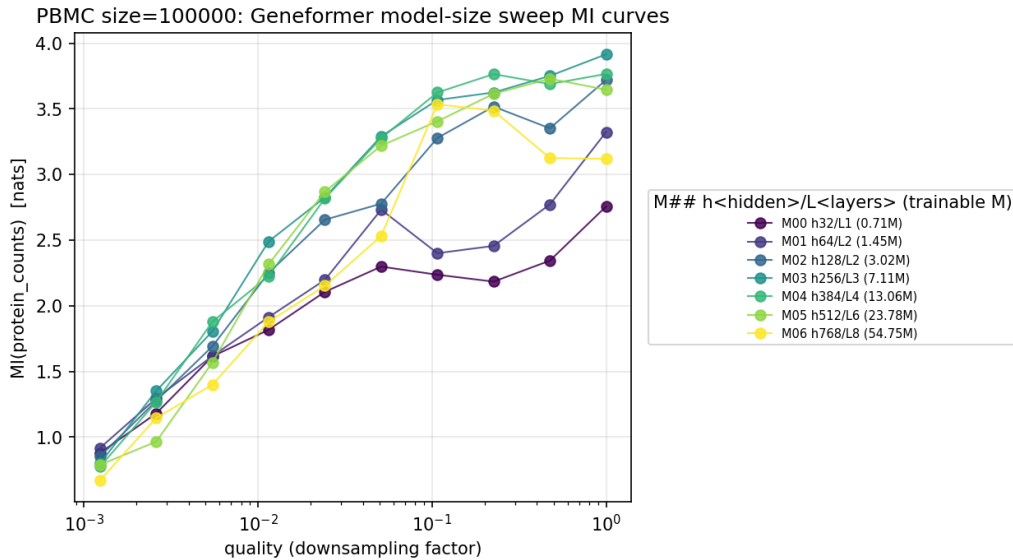


Figure 3: Geneformer MI on PBMC `protein_counts` signal vs. total trainable parameters for seven architecture configurations,

5 STATE Hyperparameter Sweep

The STATE model architecture was held fixed throughout: 256-dimensional token embeddings (`emsize= 256`), 512-dimensional feedforward layer (`d_hid= 512`), 4 attention heads, 3 transformer layers, and 256-dimensional output projection (`output_dim= 256`).

A grid search over four hyperparameters was run on PBMC at the largest cell count (100 000 cells) across all ten quality levels:

Hyperparameter	Values searched
Dropout	0.0, 0.1, 0.2, 0.3
Batch size	32, 64, 128
Peak learning rate	10^{-6} , 10^{-5} , 5×10^{-5} , 10^{-4} , 5×10^{-4} , 10^{-3}
Weight decay	10^{-4} , 10^{-3} , 10^{-2} , 10^{-1}

Table 2: STATE hyperparameter search space. Eight random configurations were sampled plus one production config (dropout 0.1, batch 64, lr 10^{-4} , wd 10^{-2}), giving 9 trials.

Every trial was trained for exactly 15 000 optimiser steps using AdamW. Validation loss was checked every 1 000 steps (limited to 50 batches per check); early stopping used patience 5. LMI was computed at the best checkpoint on the `protein_counts` signal. The figure shows one curve per HP trial: MI on the `protein_counts` signal plotted against quality (log scale) for each of the 9 configurations, allowing a direct visual comparison of how each set of hyperparameters affects MI across the full quality range.

Data: `analysis/final_results/hyperpam_sweep.csv`

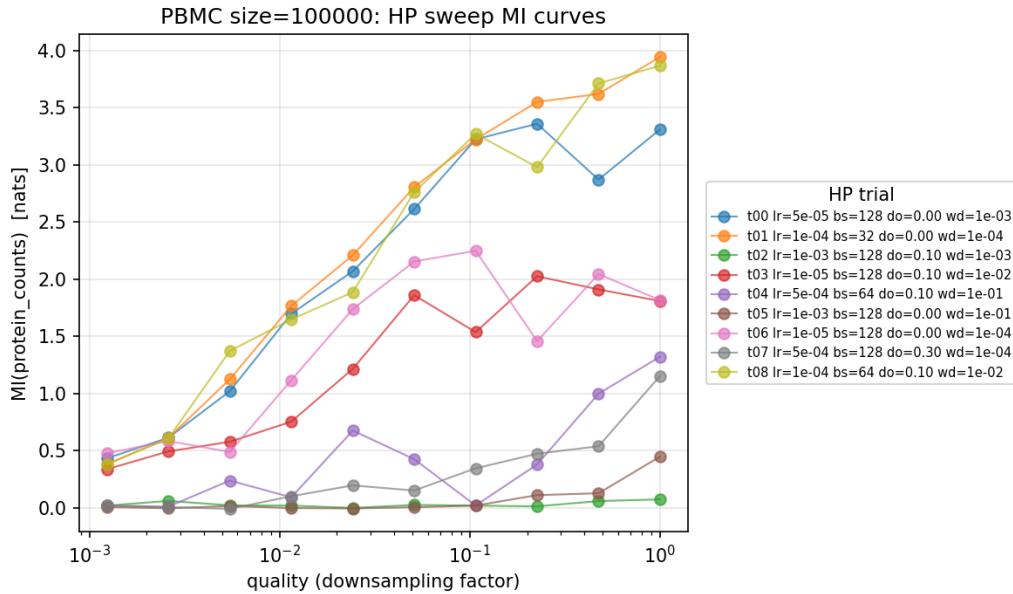


Figure 4: STATE hyperparameter sweep on PBMC (100k cells). Each line is one of the 9 HP configurations; x-axis is quality (log scale), y-axis is MI on `protein_counts` (nats).

6 MI Estimator Robustness: KSG vs. LMI

As an estimator-independent cross-check, MI was recomputed using the KSG k -nearest-neighbour estimator ($k=3$, base-2 logarithm, values in bits) on the raw 256-dimensional SCVI embeddings for the shendure dataset. KSG operates directly in the original embedding space without any learned projection or dimensionality reduction, making its assumptions entirely orthogonal to those of LMI. The `author_day` signal was one-hot encoded via `pd.get_dummies`. In a second analysis step, the saved KSG and LMI values were loaded together and compared: a scatter of KSG (bits) vs. LMI (bits, converted from nats) across all 100 (size, quality) combinations was produced with a linear fit and Pearson/Spearman correlation statistics. The final CSV and figure saved to `final_results` contain only the KSG values (column `mi_bits`), showing KSG MI in bits as a function of quality with one curve per dataset size; the KSG–LMI correlation scatter is available in the analysis notebook but not saved here.

Data: `analysis/final_results/ksg_vs_quality_shendure_SCVI.csv`

shendure / SCVI - KSG MI vs quality, one curve per size

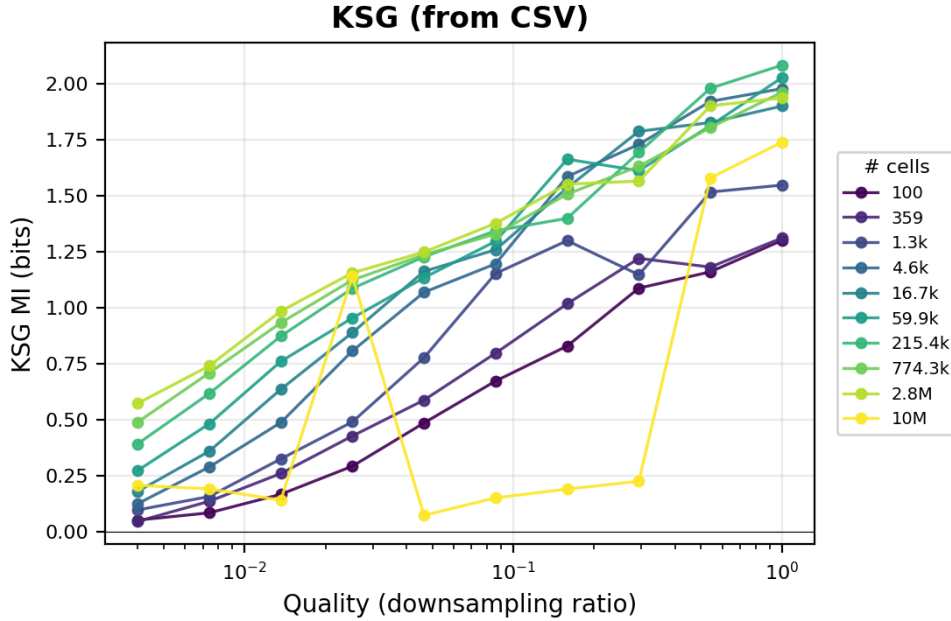


Figure 5: KSG MI (bits) for SCVI on shendure vs. measurement quality (log scale), one curve per dataset size. The KSG–LMI correlation scatter with fit statistics is in the analysis notebook.

7 Linear Probe Scaling Curves

Supervised probes were trained on top of frozen embeddings to assess downstream-task performance independently of MI. The prediction target and probe type differ by dataset:

- **PBMC** — Ridge regression ($\alpha=1.0$) predicting surface **protein_counts** (multi-output); metrics: mean R^2 , MSE, and MAE across protein channels.
- **MERFISH** — Ridge regression ($\alpha=1.0$) predicting the embedding of each cell’s nearest-neighbour in gene-expression space (**ng_idx**); metrics: mean R^2 , MSE, MAE.
- **larry** — Lineage prediction via Ridge regression ($\alpha=1.0$): embeddings of early time-point cells (days 2 and 4) are used to predict the embedding of the matched late time-point cell (day 6) sharing the same clone barcode. Clone-matched pairs are sampled once per clone; clones absent from either time point are excluded. Metrics: mean R^2 , MSE, MAE across embedding dimensions.
- **shendure** — Multi-class **author_day** classification using a k -NN probe ($k=5$) and a multinomial logistic regression (L-BFGS solver, max iterations 2000); metrics: accuracy and macro-averaged F1.

In all cases the data were split 70/30 (train/test) with seed 42, and embeddings were subsampled to at most 100 000 rows where necessary. All (algorithm, dataset, cell-count, quality) combinations were evaluated in parallel using 8 workers and assembled into a single long-format table. Scaling curves were plotted on log-log axes with one curve per algorithm, as a function of both quality and cell count, with datasets in separate panels.

Data: analysis/final_results/linear_probe_scaling.csv

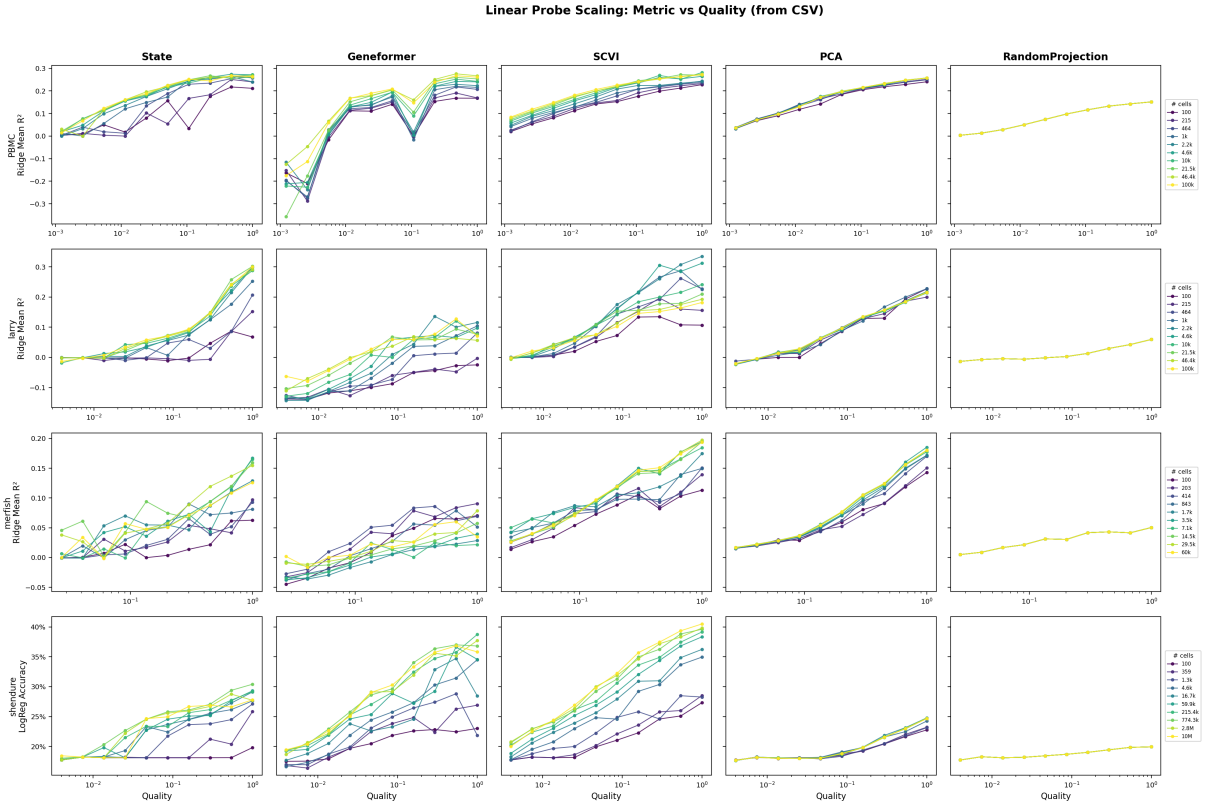


Figure 6: Linear probe R^2 scaling curves for all algorithms, plotted against quality and cell count.

8 Learning Trajectory: MI at Each Training Checkpoint

To study how MI evolves during Geneformer training on the shendure dataset, LMI was evaluated at 20 evenly-spaced checkpoints for each (size, quality) combination in the 10×10 grid. The checkpoints were identified by scanning the saved `checkpoint-*/` directories and selecting those whose step indices are evenly spaced between the first and last available checkpoint. LMI was computed with `latentmi` using a fixed random seed of 42, a maximum of 300 training epochs, and an inference batch size of 50. The signal used was `author_day`, one-hot encoded. The evaluation was run across 8 GPUs. The resulting per-checkpoint MI values were plotted as a function of training step, with one panel per dataset size and one curve per quality level within each panel.

Data: `analysis/final_results/shendure_geneformer_checkpoint_mi.csv`

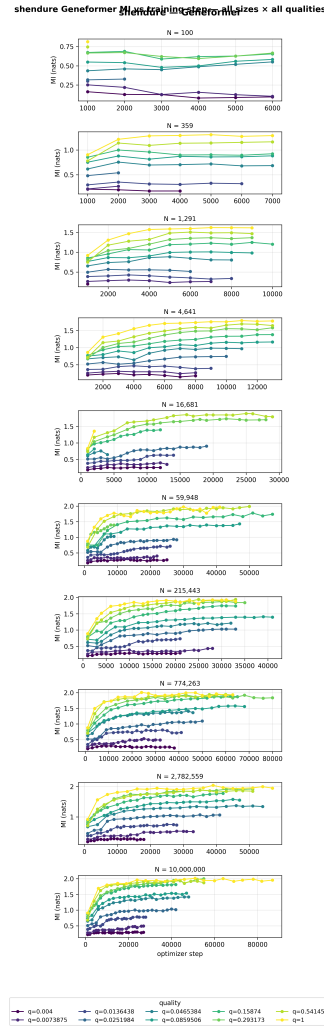


Figure 7: MI vs. training step for Geneformer on shendure. Panels correspond to dataset sizes; colours indicate quality levels.

9 Fine-Tuned Large Pretrained STATE

The publicly available SE-100M pretrained STATE model (`arcinstitute/SE-100M` on Hugging-Face) was fine-tuned on the full PBMC 10×10 grid. The pretrained weights were loaded and the optimiser state was stripped before fine-tuning began. Fine-tuning ran for 3 epochs using AdamW. The peak learning rate was scaled with dataset size:

$$\text{lr} = 10^{-6} \times \max\left(0.3, \sqrt{\min\left(1, \frac{N}{10,000}\right)}\right),$$

which gives a floor of 3×10^{-7} for the smallest datasets and caps at 1×10^{-6} for all sizes at or above 10 000 cells. This is well below the 5×10^{-4} used for training from scratch, to avoid overwriting the pretrained representations. The per-device batch size was 64 with gradient accumulation over 16 steps, giving an effective batch size of 1 024. Dropout was 0.15 for cell counts $\geq 10\,000$ and 0.2 for smaller sizes; weight decay was 0.01 for sizes $\geq 10\,000$ and 0.05 otherwise. Early stopping was disabled; training ran for the full 3 epochs. Validation loss was recorded every 1 000 steps (50 batches per check). After fine-tuning, per-cell embeddings were extracted and LMI was computed identically to all other experiments on the `celltype.13` and `protein_counts` signals. The resulting MI values were plotted alongside the from-scratch STATE results and all other algorithm baselines as noise-scaling curves on a log-log scale.

Data: `analysis/final_results/finetune_se_100m_state_pbmc_noise_scaling.csv`

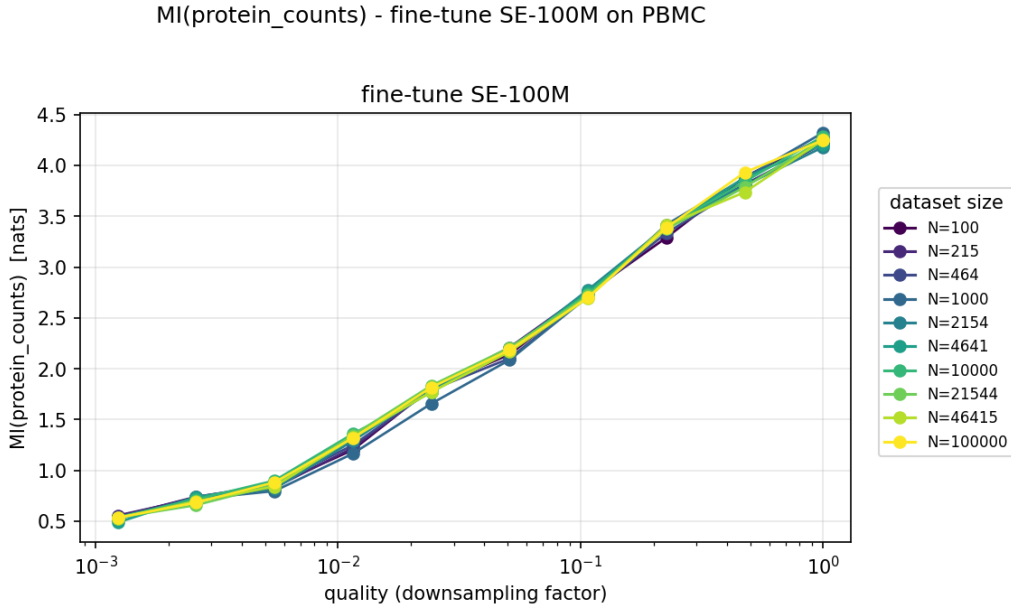


Figure 8: Noise-scaling curves for PBMC: fine-tuned SE-100M STATE vs. from-scratch STATE and other baselines.